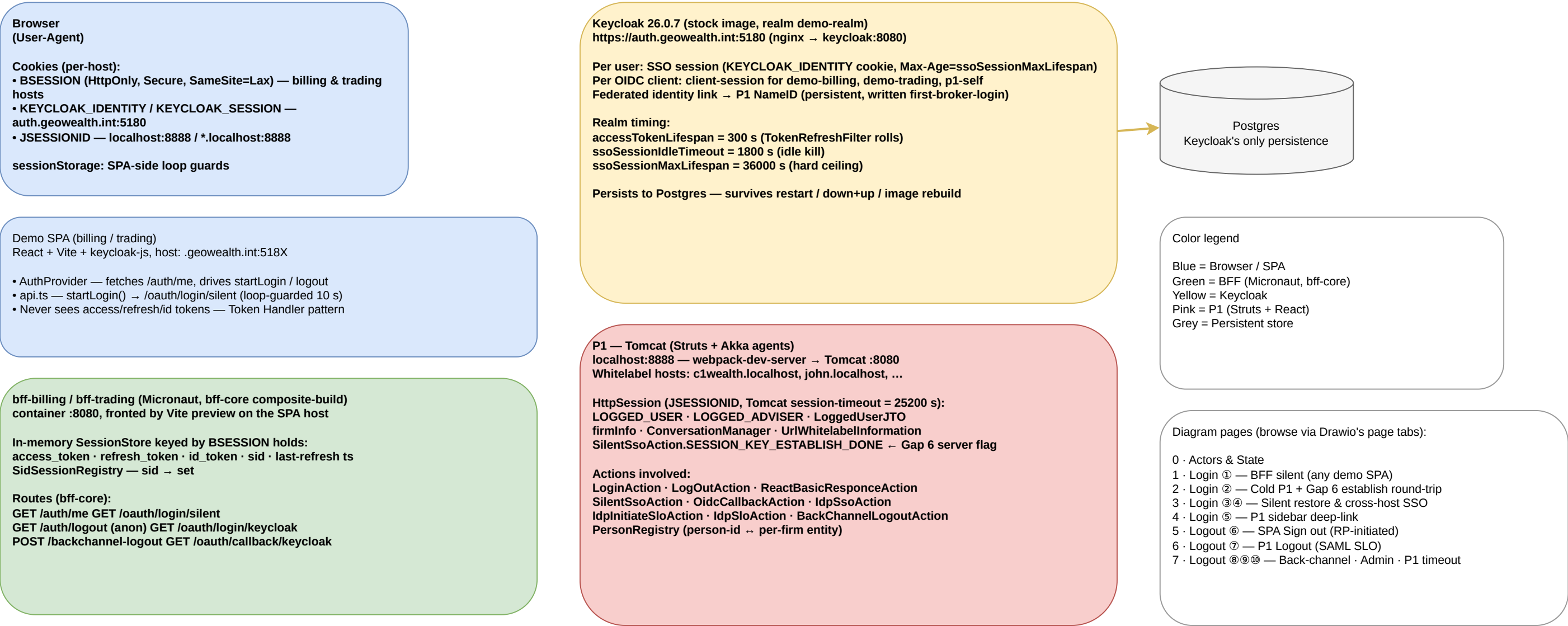


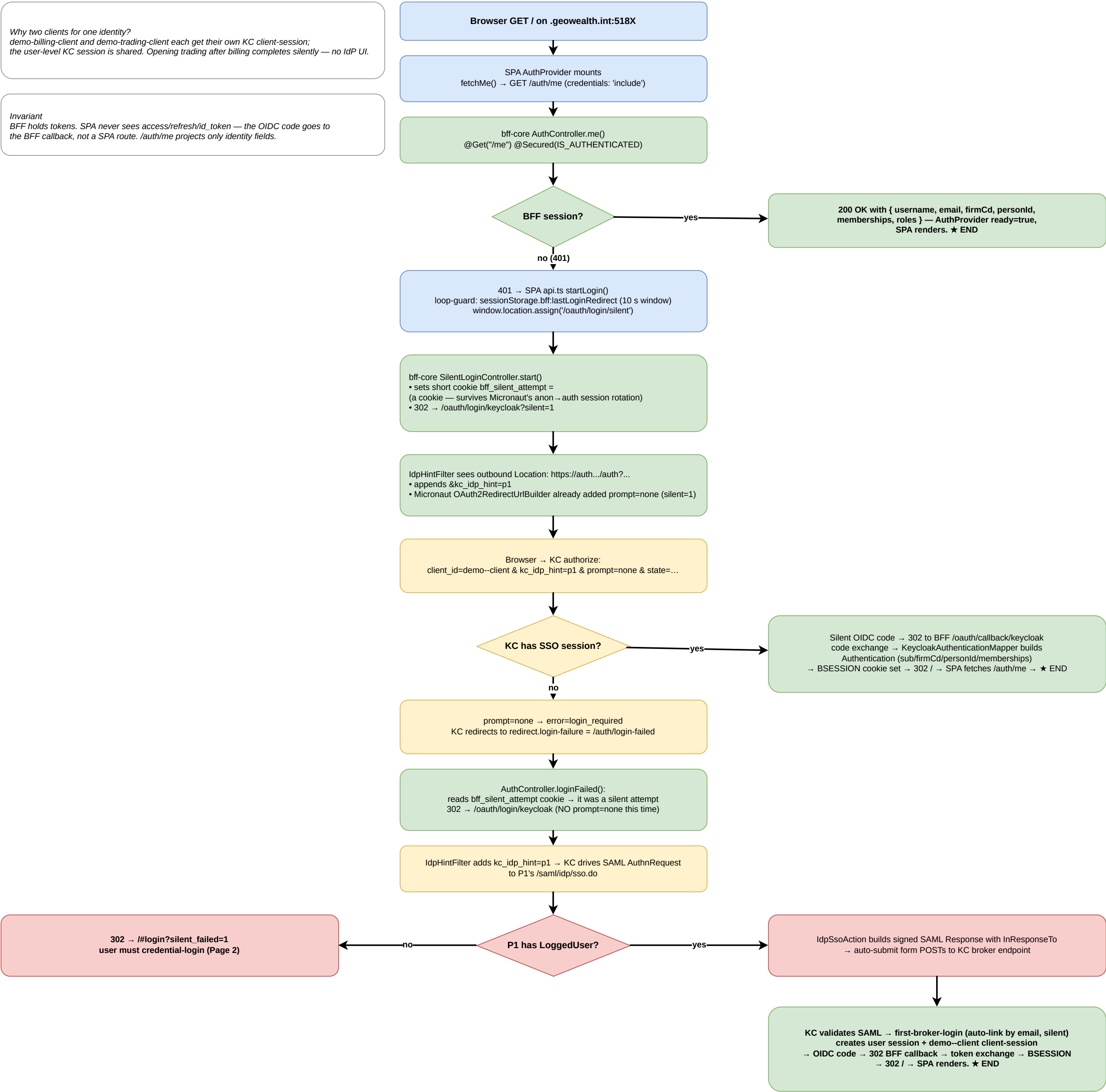
Login / Logout — actors and where state lives

Six independent state stores — KC SSO session, BFF server sessions, P1 HttpSession, browser cookies — never expire in lock-step. That gap is why Gap 6 and silent SSO recovery exist.



Login ① — BFF Token-Handler silent flow (entry: GET https://.geowealth.int:518X/)

Probe with prompt=none. If KC has a session → silent code. If not → fall through to SAML federation against P1.



Login ② — Cold P1 visit + Gap 6 establish round-trip (entry: http://localhost:8888/)

P1 credential login alone does NOT mint a KC session. Gap 6 = post-login establish=true bounce so KC has a session for downstream demo hosts.

Recovery on user reload (2026-06-04)

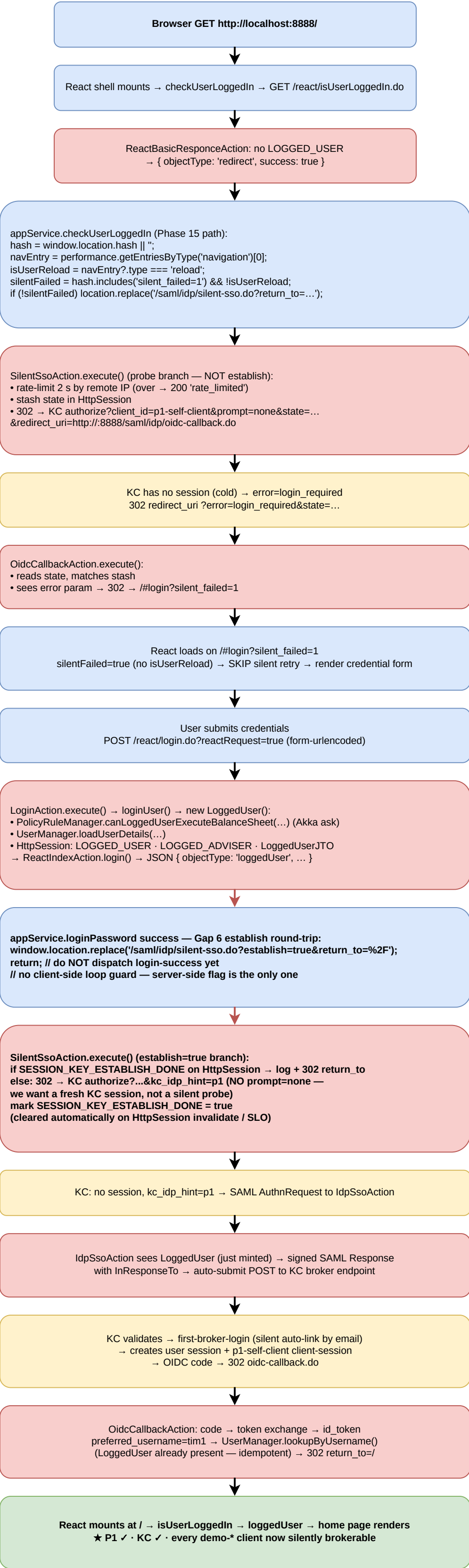
On /#login?silent_failed=1, F5 / Cmd-R produces performance.navigation.type === 'reload'. The Phase 15 guard treats isUserReload as a retry signal — silentFailed is forced false, so the silent probe re-fires and recovers SSO if KC is healthy again.

App-initiated window.location.replace(...) reports 'navigate' — the loop guard still blocks bounce storms there.

Gap 6 — why this round-trip exists

Without the establish bounce, P1 has a LoggedUser but KC stays sessionless. Next visit to billing/trading would silently probe with prompt=none, get login_required, and dead-end on credential form even though the user just logged in.

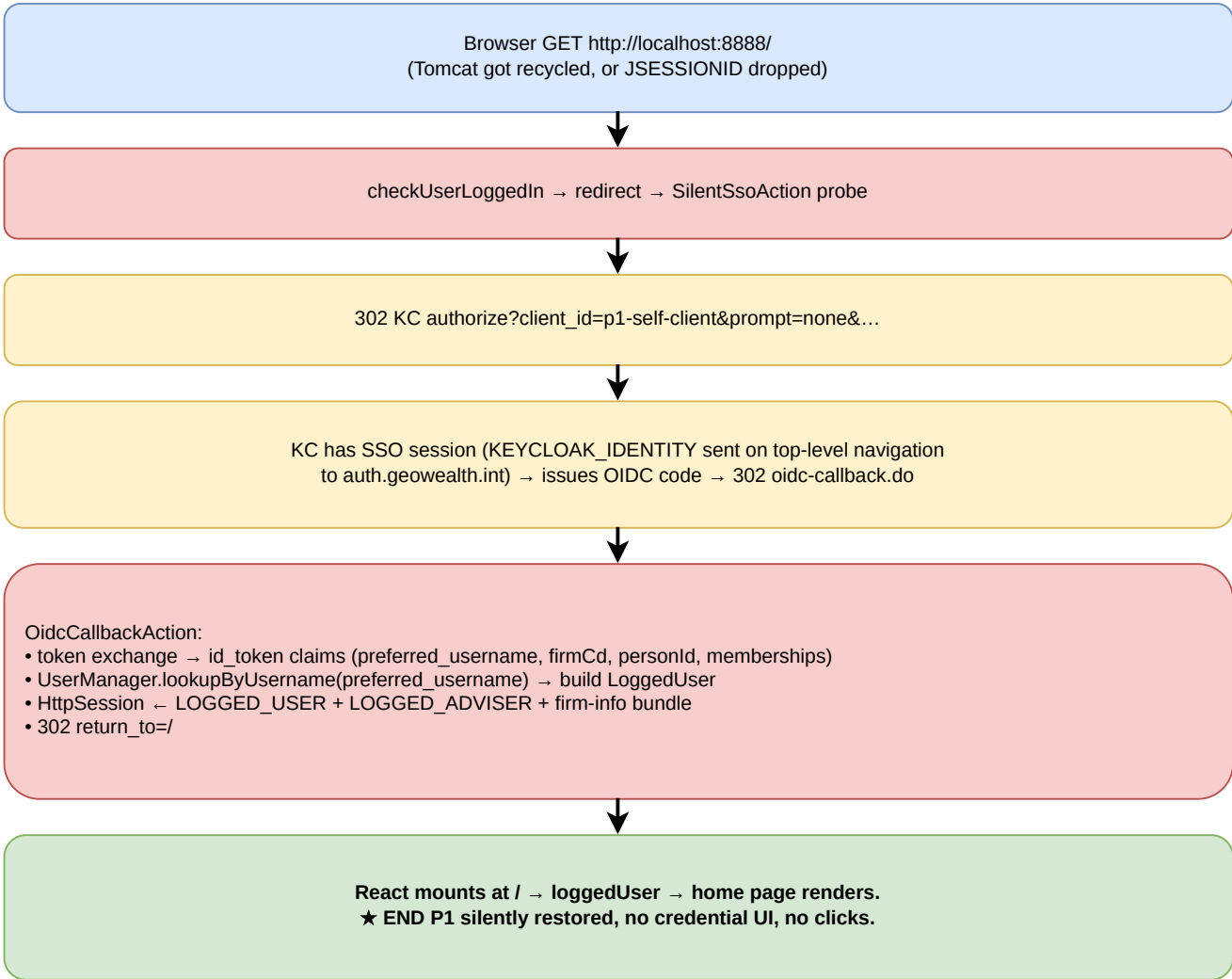
Loop prevention lives on the P1 server (SESSION_KEY_ESTABLISH_DONE). A prior client-side sessionStorage.kc_sso_established flag was removed because a stale value silently suppressed the round-trip forever on long-lived tabs after re-login.



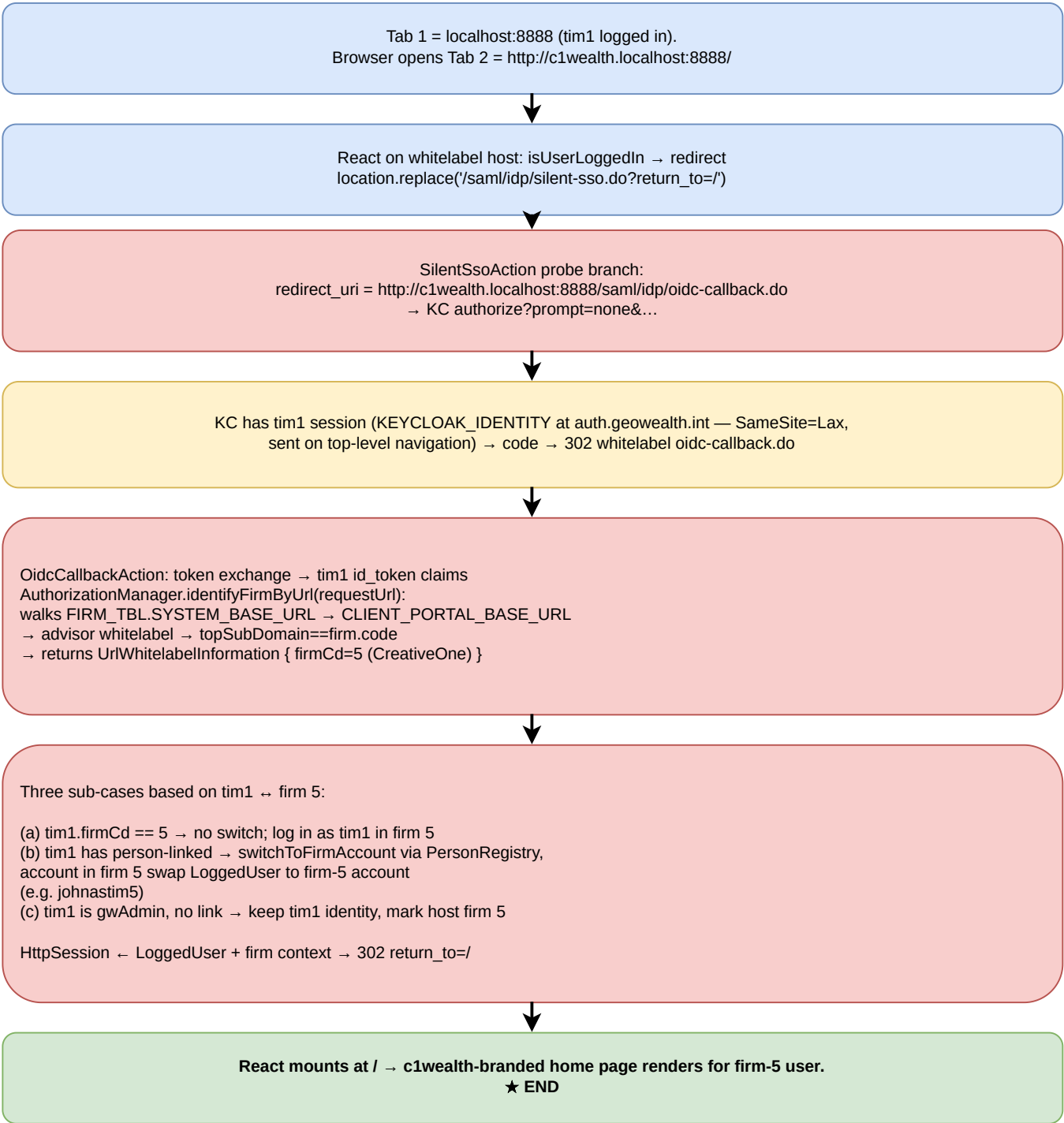
Login ③ — Silent P1 restore (KC session alive) · Login ④ — Cross-host silent SSO on whitelabel

If KC has a session, prompt=none returns a code — no credential UI. Cross-host uses the same probe but a whitelabel callback URL that triggers host-firm switch.

③ Cold P1 visit, KC session already alive



④ Cross-host silent SSO from whitelabel host

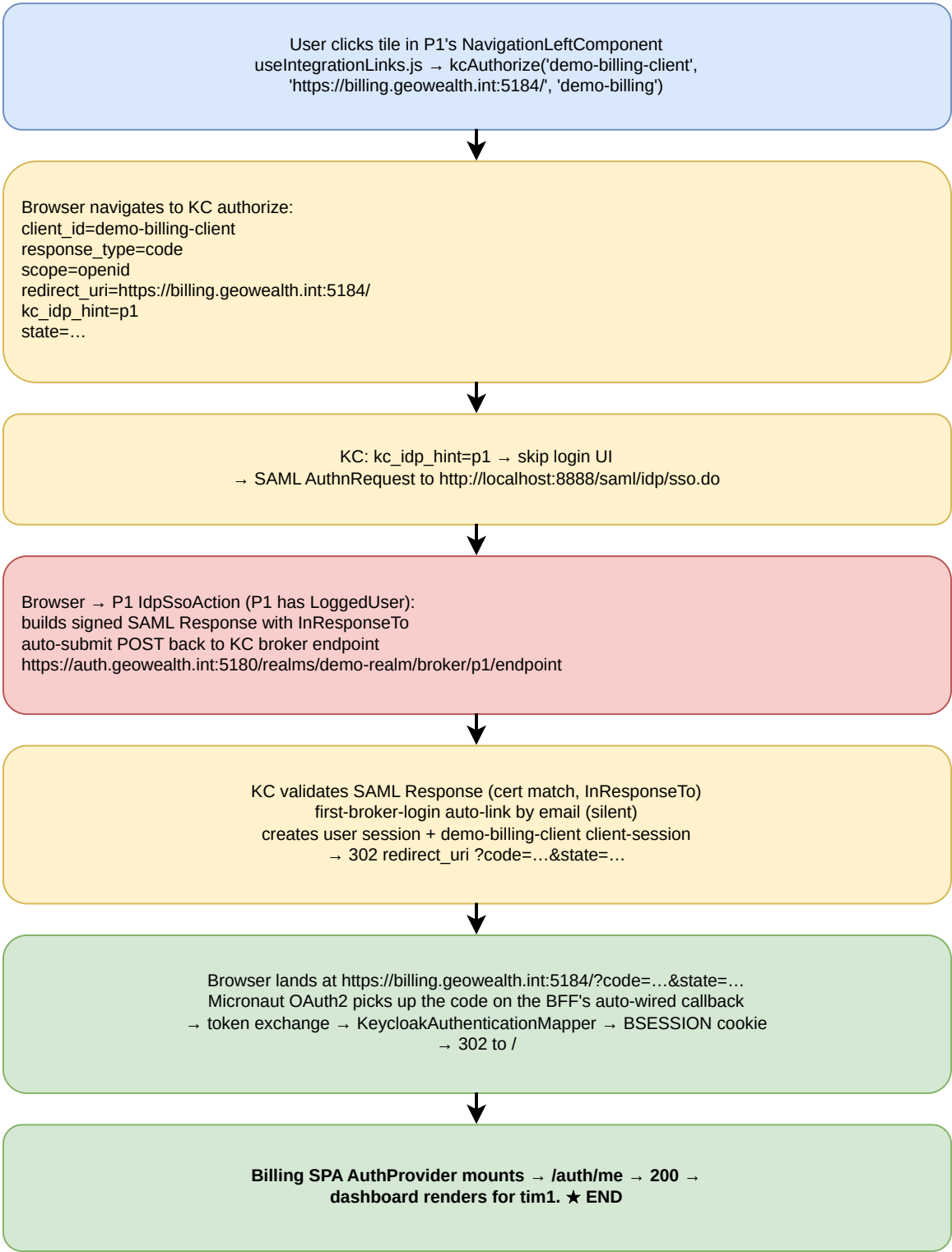


Recovery on F5 (2026-06-04)
If KC's session vanished (idle timeout / logout / restart) the probe gets login_required and falls to /#login?silent_failed=1.
A user F5 sets performance.navigation.type='reload' → isUserReload=true → silentFailed=false → probe re-fires and SSO recovers the moment KC has a session again. App bounces (window.location.replace) report 'navigate' so the loop guard still holds.

Login ⑤ — P1 sidebar deep-link into a demo SPA (SP-init, kc_idp_hint=p1, browser top-level navigation)

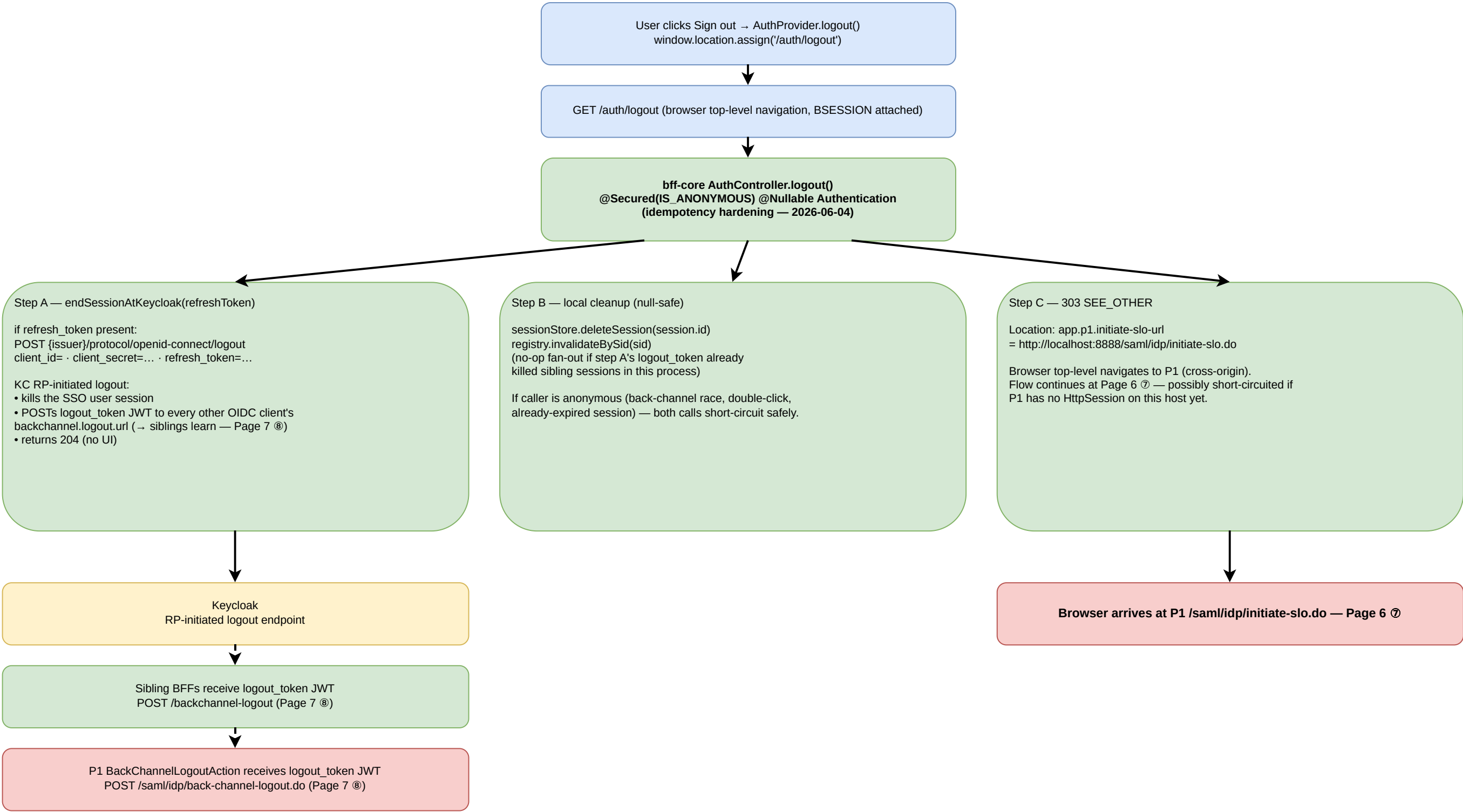
P1 sidebar tile is a kcAuthorize(...) URL — KC skips its login UI, brokers via P1's SAML IdP, then drops a code on the SPA host's BFF callback.

Why not pure IdP-init?
Pure IdP-init (P1 → KC with unsolicited Response) does NOT work cleanly against an OIDC client in KC.
The /endpoint path needs SP-init correlation; /endpoint/clients/{id} requires SAML protocol clients.
SP-init via kc_idp_hint is the supported flow.



Logout ⑥ — Sign out from a demo SPA (RP-initiated; fans back-channel logout_token to siblings)

Step A — KC end-session. Step B — local cleanup (null-safe). Step C — 303 to P1 SLO. All steps run unconditionally — idempotent under double-click / expired session / back-channel race.



What used to break (now fixed, 2026-06-04)
@Secured(IS_AUTHENTICATED) on /auth/logout turned a double-click, a back-channel race, or an already-expired session into a 401 HATEOAS error page.
The endpoint now accepts anonymous callers — worst case is best-effort cleanup with nothing left to clean, and the user still gets 303'd to P1 SLO.

Logout ⑦ — Logout link in P1 (SAML SLO)

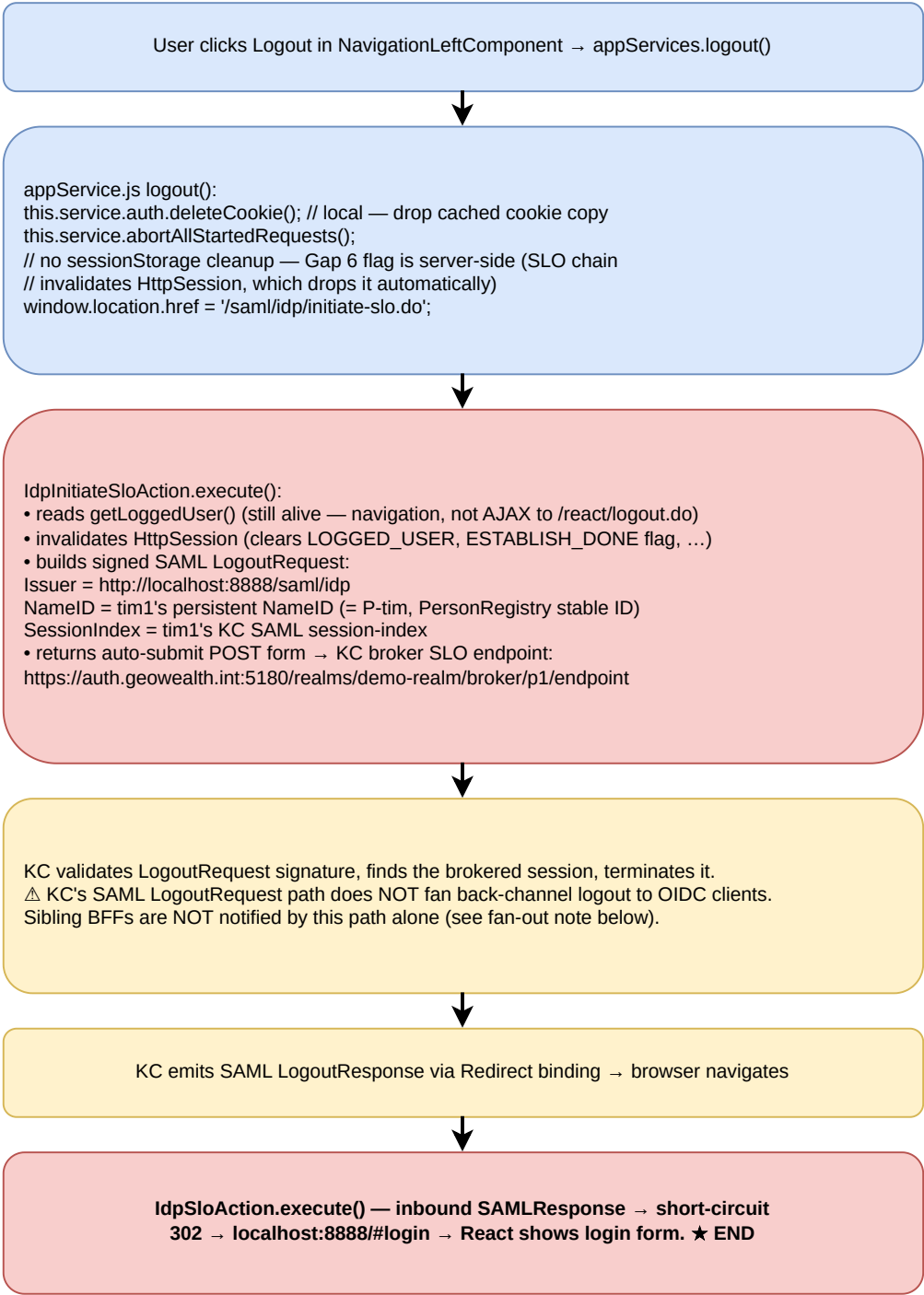
P1-initiated SAML LogoutRequest kills the KC brokered session — but KC does NOT fan back-channel logout to OIDC clients from this path (issues #17318 / #21770). Sibling BFFs hear about it via BackChannelLogoutAction on P1.

Sibling BFF fan-out for P1-initiated logout

Step ⑦ alone leaves billing/trading sessions with stale refresh tokens.
Hand-off is in KC's back-channel config:
P1 has backchannel.logout.url =
http://host.docker.internal:8888/saml/ldap/back-channel-logout.do
Each demo BFF likewise has /backchannel-logout configured.

When KC ends the SSO session (whether triggered by ⑥, ⑦, ⑧ idle, or admin),
it POSTs logout_token to every OIDC client with a back-channel URL.

P1 receives this on BackChannelLogoutAction and runs the same
sid → session invalidate dance the BFFs do (Page 7 ⑥).



Logout ⑧ Back-channel into BFF · ⑨ Admin force-logout · ⑩ P1 HttpSession timeout

⑧ How sibling BFFs hear about logouts. ⑨ How admin / idle / max-lifespan fan out. ⑩ Why P1 idle heals itself the next time the user touches the app.

⑧ Back-channel logout into a BFF

⑨ Admin force-logout / KC idle / max-lifespan ⑩ P1 HttpSession timeout

KC ends a user session (path: ⑥ ⑦ ⑨ idle, max-lifespan, admin)
Server-to-server, no browser involved.

⑨ Admin or idle / max-lifespan

POST /admin/realms/demo-realm/users/logout
POST /admin/realms/demo-realm/clients/logout-all

OR

ssoSessionIdleTimeout = 1800 s (idle)
ssoSessionMaxLifespan = 36000 s (hard ceiling)

POST /backchannel-logout
Content-Type: application/x-www-form-urlencoded
logout_token=

BackchannelLogoutController.logout(@Nullable @Body("logout_token") t)
@Secured(IS_ANONYMOUS)

- t blank → 400
- LogoutTokenValidator.validateAndGetSid(t):
parse JWT header → kid → fetch JWKS from configured jwks.url
(points at in-cluster keycloak:8080 — separate route from the SPA-facing public auth.geowealth.int)
verify: signature · exp · aud=this-client · iss=issuer
sid present · events contains
http://schemas.openid.net/event/backchannel-logout
→ return sid
- registry.invalidateBySid(sid):
walks SidSessionRegistry's in-memory sid → session-id-set map,
deletes each session from SessionStore, logs the killed count
- 200 OK

KC ends user SSO session, then fans logout_token POSTs to every OIDC client with backchannel.logout.url:

- bff-billing /backchannel-logout
- bff-trading /backchannel-logout
- P1 /saml/idp/back-channel-logout.do

Each receiver = ⑧ above. Same sid → session invalidate dance.

Next SPA call → AuthController.me → no session → 401
→ api.ts startLogin() (loop-guarded) → /oauth/login/silent → KC says
login_required → /login?silent_failed=1 (Page 2)

⑩ P1 HttpSession timeout (server-only)

Tomcat reaper invalidates P1 HttpSession after 7h idle
(session-timeout = 25200 s).
P1 has no notification path back to KC for this.

Next time the user touches P1:

- checkUserLoggedIn → redirect → SilentSsoAction
- KC still has a session (35-min idle limit, 10h max)
- probe succeeds → cold-with-KC-session path (Page 3 ③)
silently restores LOGGED_USER from id_token claims.

★ Timeouts heal themselves as long as KC is younger than P1 idle.

Cross-cutting invariants (from §4 of the algorithm doc)

- Token Handler — SPA never sees access/refresh/id_token. BSESSION HttpOnly+Secure. Code goes to BFF callback only. /auth/me projects identity fields.
- Every login brokered through P1 SAML — realm has no native users; IdpHintFilter pins kc_idp_hint=p1 on every BFF authorize; SPA only uses /oauth/login/silent.
- Rolling refresh on the BFF — TokenRefreshFilter refreshes when < 60 s of access-token left; failure invalidates session and SPA bounces through startLogin().
- Loop guards everywhere — 10 s SPA window, P1 server-side SESSION_KEY_ESTABLISH_DONE, SilentSsoAction 2 s IP rate-limit.
- sid = back-channel correlator — written into BFF session by KeycloakAuthenticationMapper; lookup in BackchannelLogoutController.
- personId stable across subdomains — PersonRegistry.resolvePersonId(homeUser) → KC OIDC user-attribute mapper.
- Cross-host silent SSO needs a KC session — Gap 6 round-trip after credential login; silent_failed loop guard honours user F5 (isUserReload).